

SEO HEE JUNG

Build it. *Validate it.* Learn it.

OS 차이 검증, 부하 테스트, 실시간 메트릭 파이프라인 구축, DevOps 인프라까지 — 직접 구현하고 검증하며 시스템을 이해한 포트폴리오

FastAPI · Linux · Ansible

Spring Boot · Resilience4j · JMeter

k6 · Kafka · Kotlin · InfluxDB

GitLab · Docker Compose · GitHub Actions · GitLab Runner · Nginx

About Me

서희정

시스템이 어떻게 동작하는지,
어떤 조건에서 한계에 도달하는지를 직접 확인합니다.
코드가 돌아가는 것보다 왜 돌아가는지를 이해하는 데 집중합니다.

관심 영역

- Backend 시스템 설계 및 안정성 개선
- 관측 가능성(Observability) — 메트릭·트레이싱·로깅
- 부하 테스트 및 성능 병목 분석
- 인프라 자동화와 CI/CD 운영 환경 구축
- OS 레벨 시스템 동작 이해

공통으로 추구하는 것

"동작하는 코드"가 아닌 "왜 동작하는지 아는 코드"

직접 검증하고 · 수치로 증명하고 · 문서로 남긴다

프로젝트별 핵심 성과

01	OS 환경 차이 3단계 직접 검증
02	Circuit Breaker OPEN 조건 실증
03	Kafka consumer lag 83.2% 감소
04	온프레미스 DevOps 운영 흐름 구축

FastAPI · Linux · Ansible

서버 상태 모니터링 대시보드



목적

- 개발 환경(Windows)과 운영 환경(Linux) 간 서버 모니터링 동작 차이를 직접 구현·실행·오류 경험을 통해 검증하기 위한 프로젝트
- 단순 수치 나열이 아닌 "**이 서버에서 무엇이 실행 중이며, 지금 안전한 상태인가**"를 설명하는 운영 판단형 대시보드

Windows에서 정상 동작하던 시스템 수집 로직이 Linux에서는 권한·OS API·서비스 구조 차이로 동일하게 동작하지 않음을 실증적으로 검증.

구성

- Python : FastAPI 백엔드, psutil 기반 시스템 리소스·프로세스 수집
- HTML/CSS/JS : 관리자 대시보드 UI
- YAML/Jinja2 : Ansible Playbook, systemd 유닛, 환경 변수 템플릿 구성

검증 포인트

- OS 차이 3단계 직접 검증
- Ansible + Ansible Vault 기반 운영 자동화
- Rocky Linux Native 최종 검증

기획 의도: 운영자가 판단할 수 있는 대시보드

검증 질문

- Windows에서 정상 동작하던 수집 로직이 Linux에서도 동일하게 동작하는가?
- 권한, OS API, 서비스 구조 차이가 어떤 오류로 드러나는가?
- 운영 환경에서 systemd, journalctl, psutil 접근은 어디까지 가능한가?

MongoDB 역할 분리

- 종료 판단의 필수 의존성이 아님
- 이름 기반 정적 정책 저장소
- 실시간 psutil 판단이 최종 기준
- UI 용 종료 사유 텍스트 제공

개발 및 검증 프로세스 흐름

Step 1: Windows 개발 (FastAPI 대시보드)

- FastAPI를 활용하여 대시보드 기능 구현
- psutil을 사용하여 CPU, 메모리, 디스크, 프로세스 정보에 대한 검증 수행

Step 2: Docker Rocky Linux

- 동일한 코드를 Docker Rocky Linux 환경에서 실행하여 1차적인 환경 이식성 확인

Step 3: OS 간 차이점 분석 (Docker Rocky Linux와 Windows 환경)

- psutil 권한 영향 차이
- systemctl 호출 가능 여부
- 네트워크/포트 권한 제한
- 로그 · 리소스 접근 방식 차이

Step 4: Native Rocky Linux (최종 전체 검증)

- systemd, 로그, 권한 설정을 포함한 전체 시스템에 대해 종합적으로 검증 완료

주요 기능: 수집보다 해석이 핵심

CPU/RAM/Disk 수집에 그치지 않고, 프로세스의 위험 여부와 종료 가능 여부까지 판단

① 시스템 리소스 수집

- CPU / Memory / Disk / Uptime 실시간 조회
- psutil 기반 OS 레벨 정보 수집

② 서비스 상태 모니터링

- systemctl 기반 서비스 상태 조회 (Linux)
- active/inactive/failed 판단
- Windows는 일부 제한

③ 프로세스 분석

- 실행 중 프로세스 목록 · PID · 실행 경로 · CPU/메모리 · 실행 유저 · 열린 포트 수집
- 프로세스 목록 시각화

④ 정책 기반 해석

- MongoDB/JSON KNOWN_PROCESSES 정책으로 정상/일반/의심 분류
- 종료 기능 + 인증/인가 포함

위험 판단 기준 & 종료 정책 상세

Warning Rules (위험 레벨 기준)

Rule	조건	레벨
RUNNING_AS_ADMIN	root/SYSTEM/Administrator 실행 (PID 4 제외)	● WARN
PUBLIC_PORT(n)	KNOWN_PORTS 주요 서비스 포트 사용	● / ●
SYSTEM_PORT(n)	port < 1024 (KNOWN_PORTS 제외)	● / ●
HIGH_MEMORY_USAGE	메모리 사용률 ≥ 20%	● INFO
SUSPICIOUS_PATH	표준 경로 외 실행 (치명 Warning)	● DANGER

치명 Warning (SUSPICIOUS_PATH, RUNNING_AS_ADMIN) 하나라도 존재 → DANGER 판정

종료 판단 플로우

1. 사용자 종료 요청
2. 실시간 psutil 재수집
3. MongoDB known_processes 조회 (연결 시)
4. 시스템 프로세스 여부 확인
5. Soft Kill (SIGTERM, 최대 5초 대기)
6. Hard Kill (SIGKILL, 실패 시만)

상태 판정 기준

상태	조건
OK	KNOWN_PROCESSES 등록 + Warning 없음
WARN	UNKNOWN or 일반 Warning 존재
DANGER	치명 Warning 하나라도 존재

종료 차단 대상

OS	차단 대상
Windows	SESSION 0, SYSTEM/LOCAL/NETWORK SERVICE, Windows signed core binary
Linux	UID 0 (root), systemd/daemon, /usr/lib/systemd

Ansible 기반 운영 환경 재현

Ansible 수행 핵심 작업 흐름

STEP 01 : Git Clone

/home/rockylinux/server-monitor/ 경로로 코드 다운로드

STEP 02 : 운영 디렉터리 세팅

/opt/server-monitor/ 생성 후 web/app/ 복사 배치

STEP 03 : Python venv 환경 구성

/opt/server-monitor/venv/ 생성 + requirements.txt 패키지 설치

STEP 04 : systemd 서비스 등록

FastAPI 앱 systemd 유닛 배치 + 자동 시작 기반 구성

STEP 05 : MongoDB 인증 + Vault 비밀 값 분리

MongoDB 인증 활성화 + Ansible Vault 기반 민감 정보 분리

디렉토리 역할 분리

- /home/rockylinux/server-monitor/ : Git 레포 전체 clone
- /opt/server-monitor/ : 운영 전용
- app/ · venv/ · logs/ · data/ 분리
- Ansible은 패키지, 유저, 권한, 방화벽, SSH, systemd 담당

서비스 배포 도구가 아니라 “서버 1대 재설치 → 동일 상태 복구”를 위한
운영 환경 재현 도구로 설계

OS 차이에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : psutil 프로세스 수집 권한 차이

Windows에서는 정상 수집되던 프로세스 정보가 Linux에서 `PermissionError` 발생.
root 권한 없이 타 유저 프로세스의 open files, connections 수집 불가.

→ 예외 처리 분기 + 권한별 수집 가능 필드 명시적 분리로 해결

ISSUE 02 : systemctl — Docker 컨테이너에서 미 동작

Docker 컨테이너 내부에는 systemd가 없어 systemctl 호출 시
`Failed to connect to bus` 오류.

→ 환경 감지 후 컨테이너에서는 psutil 프로세스 검색 방식으로 풀백, Native Linux에서만 systemctl 사용

ISSUE 03 : MongoDB 인증 활성화 후 연결 실패

Ansible로 MongoDB 인증 활성화 적용 후 기존 연결 문자열 모두 인증 오류.

→ Ansible Vault로 username/password 분리 관리, 환경 변수 주입 방식으로 하드코딩 제거

핵심 학습 포인트

- 같은 코드, 다른 동작

OS API, 권한 모델, 서비스 구조가 달라지면 동일 코드가 전혀 다르게 동작함을 직접 검증

- 단계별 이식 검증의 중요성

Windows → Docker Linux → Native Linux 순으로 차이를 단계별 격리해야 원인 특징이 가능

- 운영 자동화는 재현성

Ansible은 배포 도구가 아니라 "서버 1대 재설치 후 동일 상태 복구"를 위한 재현 도구로 설계

- 모니터링의 목적

수치 나열이 아닌 운영자가 판단을 내릴 수 있는 해석 정보 제공이 핵심

관리자 전용 인증 시스템

인터넷 공개 서비스가 아니라 내부 관리자 도구 전제로 설계

인증 흐름

- 클라이언트 → X-API-Key 헤더 → /api/auth
- 서버: 환경 변수 ADMIN_API_KEY와 비교
- 성공 시 admin_session 쿠키 발급(HttpOnly)
- 이후 요청은 API Key 없이 쿠키 기반 인증 유지

JWT 미사용 이유

- 외부 사용자 인증 시스템 아님
- 토큰 갱신·서명 검증·만료 관리 불필요
- 단일 관리자 환경에서 오버 엔지니어링
- 서버 메모리 기반 세션 구조 선택

코드 구조

- **routes/auth.py** : 인증/세션
- **routes/dashboard.py** : 리소스 수집 API
- **routes/process.py** : 프로세스 분석/종료
- **services/system** : cpu, memory, disk, process, service, log
- **constants** : linux.py, windows.py, ports.py

성능 포인트

- process_iter(attrs=...) 사용
- proc.oneshot()으로 시스템 콜 비용 감소
- AccessDenied는 cpu/memory None 처리

02 / 04 · Circuit Breaker

Spring Boot · Resilience4j · JMeter

부하 대응 및 관측 시스템

 GitHub

목적

- 정상 응답보다 실패가 누적될 때의 상태 전이와 방어 흐름을 관측하는 데 초점을 둔 백엔드 실험 프로젝트.
- CPU 포화·DB 커넥션 고갈 등 장애를 API로 재현하고 Rate Limit·Circuit Breaker 트리거를 실증 검증.

극한 부하에서 시스템 반응을 정밀 계측하고, Rate Limit과 Circuit Breaker가 어떤 조건에서 동작하는지 실증한 테스트 전용 백엔드.

구현 언어/파일 역할

- Java : Spring Boot 기반 부하 재현 API, Rate Limit, Circuit Breaker, Metrics
- YAML : Spring 설정, Docker Compose, Prometheus 구성
- SQL : PostgreSQL 테스트 데이터 조회 및 DB 상태 검증

테스트 불변 설정

50%CB failureRate
Threshold**30**CB sliding
WindowSize**50**HikariCP
maxPoolSize**5s**Connection
Timeout

기획 의도: 부하 상황에서 서비스가 어떻게 보호되는지 검증

2단계 방어 체계

- 요청 인입
- 1차 방어: IP 기반 Rate Limit — 반복 트래픽 차단
- 2차 방어: Resilience4j CircuitBreaker
- CLOSED → HALF_OPEN → OPEN 상태 전이
- slidingWindowSize 30, failureRate 50%, 15건 실패 시 OPEN

반복 트래픽을 먼저 Rate Limit에서 차단하고,
통과한 요청은 Resilience4j Circuit Breaker가 상태 전이로 보호한다.

개발 및 검증 프로세스 흐름

1. Vault로 DB 자격 증명 주입
2. Docker로 로컬/운영 차이 제거
3. 부하 생성·방어 로직 Service 계층 집중
 - └ AOP로 관측 로직 분리
4. TraceldFilter → trace_id 전파
 - └ Filter → Service → DB 전 구간
5. jmeter-traffic-generator 외부 레포 기반 비정상 트래픽 주입
6. API + 부하로 CB OPEN 상황 유도
7. Prometheus + Snapshot API 검증

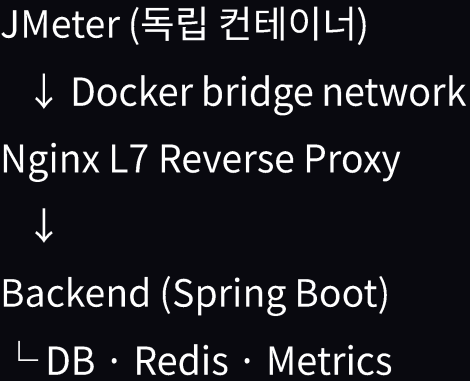
JMeter 트래픽 구성

JMeter 시나리오는 별도 jmeter-traffic-generator 저장소 기반이며, 백엔드는 부하를 수신하고 방어·관측 로직을 검증하는 대상 서버 역할을 수행

Attack vs Normal 트래픽 분리

항목	Attack-TG	Normal-TG
목적	부하 유발	가용성 확인
Thread 수	300→500→750→1000	25
Delay	100~300ms	1000ms
IP (X-Forwarded-For)	10.10.10.10	20.20.20.20

JMeter 아키텍처



실험 제약 조건

- Thread 수만 단계적으로 증가
- ON/OFF 비교는 동일 단계에서만 수행
- Resource Exhaustion Test는 repeatCount와 Pool Size 조정
- 서로 다른 실험 목적은 직접 비교하지 않음

부하 시나리오와 비교

CPU Load

Busy-loop 연산으로 실제 CPU 점유율 상승 유도

DB Read

SELECT 반복 → 커넥션 풀 고갈 · 지연 재현. 점진적 병목

DB Write

INSERT 반복 → 락 경쟁 · Commit 비용 증가. 급격한 병목

Cache ON/OFF

Redis 캐시 적용 여부에 따른 DB 부하 변동 실시간 관측

시나리오 비교

	DB READ	DB WRITE
API	/db-read	/db-write
병목 위치	Connection Pool	Lock / Commit
장애 양상	점진적 지연	급격한 병목

Observability 설계

trace_id 기반 요청 추적

모든 요청에 고유 **trace_id** 부여 (TraceIdFilter)

↓ MDC를 통해 로그 전파

Filter → Service → Repository 전 구간 추적

↓ 응답 헤더까지 일관 전파

AOP 기반 로깅 아키텍처

- LogAspect를 단일 진입점으로 사용
- LOAD_START / END / FAIL / ERROR 이벤트 체계
- JSON Logback 구조화 로그 설계

Custom Metrics (Prometheus)

- rate_limit_blocked_total : Rate Limit 차단 건수
- rate_limit_allowed_total : 허용 건수
- resilience4j.circuitbreaker.* : CB 상태
- hikaricp.connections.* : 풀 가용량
- http.server.requests : 요청 시계열

기술 스택 상세

분류	기술
Language	Java 17 (Eclipse Temurin Alpine)
Framework	Spring Boot 3.4.2
Resilience	Resilience4j Count-based Circuit Breaker
RDBMS	PostgreSQL 16
Cache	Redis 7 Primary Cache / Caffeine Bucket Store
Metrics	Micrometer + Prometheus
Tracing	MDC + TraceIdFilter
Secret	Spring Cloud Vault / HashiCorp Vault (AppRole)
Deploy	Docker Multi-stage build

부하 상황에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : CB가 예상 시점보다 늦게 OPEN

HikariCP의 connectionTimeout(5초)이 먼저 발생하면서, 해당 요청들이 실패(Exception)가 아닌 지연(Slow Call)으로 집계되어 failureRate 도달이 늦어짐.
→ CB 실패 정의(exception vs slow call)와 풀 타임아웃 타이밍을 명확히 분리해 설정

ISSUE 02 : Normal-TG까지 Rate Limit에 걸림

공격 트래픽과 정상 트래픽이 동일한 Docker Bridge 네트워크를 사용해 서버에서 식별하는 소스 IP가 동일하게 인식됨.
→ X-Forwarded-For 헤더로 IP를 명시적으로 분리(10.10.10.10 / 20.20.20.20)하고, Rate Limit 판별 로직을 해당 헤더 기반으로 수정.

ISSUE 03 : Vault 연동 후 앱 시작 시 자격 증명 로드 실패

드(application.yml)는 AppRole 인증으로 설정되었으나, 가이드 문서(setup.md)는 로컬용 Token 방식을 안내하여 혼선 발생.
→ 배포 기준은 AppRole 설정을 따르도록 명시하고, 로컬 개발 시 Token 방식을 병행할 수 있도록 가이드 문서를 분리하여 업데이트.

핵심 학습 포인트

- **장애는 설계로 제어**
문제가 생긴 뒤 대응이 아니라 임계치를 수치로 정의하고 의도적으로 트리거 해야 검증 가능
- **불변 설정의 중요성**
실험 중 설정값 변경 시 어떤 변수가 결과를 바꿨는지 분리 불가.
CB/HikariCP/Redis TTL을 고정해야 재현성 확보
- **trace_id 전파의 실용성**
장애 발생 시 Filter → Service → DB 구간 지연을 trace_id 하나로 추적 가능.
실제 장애 분석 시간 단축에 직결
- **2단계 방어의 의미**
Rate Limit(IP 기반)이 없으면 Normal-TG까지 CB에 걸려 정상 사용자도 차단됨.
계층별 방어가 필수

TraceId 전파와 AOP 로깅 구현

관측 로직은 비즈니스 로직에서 분리하고, 필터와 AOP의 실행 순서를 명시해 trace_id가 모든 로그에 남도록 구성

RuntimeFeatureState — 토크 저장소

- RateLimit ON/OFF 런타임 상태 저장
- Redis Cache ON/OFF 런타임 상태 저장
- 서버 재시작 시 application.yml 기준 초기화
- SystemSnapshot에 현재 상태 반영

TraceIdFilter

- X-Trace-Id 헤더가 있으면 재사용, 없으면 UUID 생성
- MDC.put으로 로그 컨텍스트에 저장
- 응답 헤더에 X-Trace-Id 반환
- finally에서 MDC.clear로 스레드 오염 방지

FilterOrderConfig

- TraceIdFilter order=1
- RateLimitFilter order=2
- RateLimitFilter가 항상 trace_id 이후 실행되도록 보장

LogAspect

- LoadScenarioService, CircuitBreakerTestService 대상
- LOAD_START / LOAD_END / LOAD_FAIL / LOAD_ERROR 이벤트 기록
- durationMs와 trace_id를 함께 기록

k6 · Kafka · Kotlin · InfluxDB

스트리밍 기반 부하 테스트 파이프라인



목적

- 부하 테스트 메트릭이 생성되는 순간부터 수집·저장·시각화·병목 분석까지 이어지는 관측 파이프라인 구축.
- 단순 리포트 생성이 아니라 실시간 관측과 병목 분석이 목적.

k6 부하 테스트 메트릭을 Kafka로 스트리밍하고, Kotlin Collector가 InfluxDB v3에 적재한 뒤 Grafana로 시각화하는 파이프라인.

검증 성과

- end-to-end 파이프라인 검증
- Collector → InfluxDB write 병목 위치 특정
- INFLUX_BATCH_SIZE 500 → 2000 변경으로
Kafka lag 83.2% 감소

핵심 기술

- k6 + xk6-output-kafka
- Apache Kafka, Kotlin Collector
- InfluxDB v3, Grafana provisioning
- Docker Compose 기반 로컬 관측 스택

기획 의도: 메트릭이 생기는 순간부터 분석까지

구현 언어 / 구성 파일 역할

- **Kotlin** : Kafka Consumer, InfluxDB line protocol 변환 · 적재
- **JavaScript** : smoke / load / stress k6 시나리오
- **SQL** : InfluxDB v3와 Grafana 조회 쿼리
- **YAML** : Kafka, InfluxDB, Grafana Docker Compose 구성

설계 기준

- 서비스 성능과 메트릭 파이프라인 성능을 분리
- burst traffic을 Kafka가 완충
- Collector 내부 상태를 InfluxDB에 동일 경로로 저장

병목 탐색 프로세스

01 탐색

stress 실행 후 Grafana / CLI로 lag 확인

02 가설

Collector → InfluxDB
write 처리량 부족

03 격리

batch size 단일 변수만
변경

04 재측정

동일 stress 조건 재실행

05 증명

Kafka lag 83.2% 감소
확인

경로 분리가 관측의 전제

장애가 발생했을 때 API 문제인지, Kafka/Collector/InfluxDB 문제인지 격리하기 위한 설계

경로 분리 설계

Load Path

- k6 → HTTP Load → Kotlin API
- latency / error / status 결정
- 서비스 응답 성능을 측정하는 경로

Metrics Path

- k6 → Kafka topic: k6-metrics
- Kotlin Collector → InfluxDB v3
- Grafana에서 메트릭과 Collector 상태 확인

파이프라인 흐름

1. k6 Load Generator → Kotlin API Test Target
2. k6 metrics → Kafka topic k6-metrics
3. Kotlin Collector → consume / normalize / batch
4. InfluxDB v3 → line protocol write
5. Grafana → dashboard

k6는 InfluxDB v3 직접 연동이 기본 지원되지 않아
xk6-output-kafka 커스텀 빌드로 Kafka 경유 스트리밍 구조를 구성

Kafka는 완충재이자 경계선

producer인 k6와 consumer인 Collector를 비동기로 분리해 burst traffic을 흡수하고, Collector 장애 후에도 재처리 여지를 남김

Kafka 도입 이유

- producer(k6) / consumer(Collector) 비동기 분리
- burst traffic 완충: 장애 격리와 재처리 여지 확보
- 실시간 스트림 처리 구조 확보

Kafka listener 설정 핵심

- 수동 producer/consumer 성공 ≠ k6 연결 성공
- k6는 별도 컨테이너이므로 broker 주소 해석이 다름
- advertised.listeners를 내부/외부로 분리

분리한 주소

- **PLAINTEXT** : //kafka:9092
— Docker 내부
- **PLAINTEXT_HOST** :
//host.docker.internal:29092
— 외부/호스트
- k6 컨테이너 연결 실패를 listener 분리로 해결

Kotlin Collector 설계

Collector는 단순 전달자가 아니라, k6 JSON을 InfluxDB v3에 적합한 line protocol로 변환하고 cardinality를 통제하는 중간 처리 계층

Kotlin Collector 도입 이유

- 저장 전 필드 정규화 레이어 확보
- URL full string → path 기준 정규화
- batch write 최적화 및 실패 시 재시도
- high cardinality tag 사용 제한

InfluxDB Schema

Tag : scenario / method / status / metric_name / group

Field : duration / count / value / vus

URL full string → path 기준 정규화

high cardinality tag 제한

Kotlin Collector 처리 전략

Kafka poll 단위 수신

→ k6 JSON 파싱

→ URL path 정규화

→ InfluxDB line protocol 변환

→ batch flush (**1초 or 건수 기준**)

→ write 성공 시 **commitSync()**

→ write 실패 시 offset 미커밋

Collector 구현 포인트

Offset Commit 전략

- enable.auto.commit = false
- record별 JSON → line protocol 변환
- writer.add(line) 성공 여부 확인
- allSucceeded일 때만
consumer.commitSync()

Retry / Dead-letter

- max-retries: 3
- backoff: 1s → 2s → 4s
- 재시도 대상: HTTP 429 / 5xx
- 제외: 400 / 401 / 403,
최종 실패는 dead-letter 기록

write 성공 후에만 offset을 commit해 InfluxDB 실패 시
Kafka가 동일 메시지를 다시 전달하도록 설계

Grafana 지표와 CLI 지표 분리

Kafka lag는 Collector가 직접 수집한 값이 아니라 Kafka consumer group CLI 기준으로 별도 확인

RPS

부하 정상 유입 확인

p95

지연 급증 구간 확인

Failed

장애 발생 시점 확인

VUs

시나리오 반영 여부 확인

TPS

Collector 처리량 확인

flush

buffer_size /
last_flush_ms 확인

Grafana 시각화 지표

- RPS: 부하 정상 유입 확인
- Response Time avg / p95: 지연 급증 구간
- Failed Requests: 장애 발생 시점
- VUs: 시나리오 반영 여부

Collector Stats

- tps: 초당 처리량
- buffer_size: 현재 버퍼 크기
- last_flush_ms: flush 소요 시간
- processed_total / failed_total

검증 시 주의

- kafka_lag = 0은 placeholder
- lag 판단은 Kafka consumer group CLI 기준
- Collector 내부 지표와 CLI lag를 함께 봐야 병목 위치 특정 가능

Collector → InfluxDB 병목 분석 및 개선

83.2%

Kafka consumer lag 감소율
(batch size 500 → 2000)

lag 감소율 계산식

$$(1,212,689 - 203,648) / 1,212,689 \times 100 = 83.2\%$$

지표	Before	After
Kafka lag	1,212,689	203,648
avg latency	4~5ms	~3.2ms
RPS	~10	~10
Failed Req	0%	0%
Response Time	Grafana 지연	정상 표시

문제 상황

- stress 조건: 100 → 200 → 500 VUs
- Collector batch size: 500
- batch write: 70KB~75KB
- flush: 약 990ms

개선 적용

- INFLUX_BATCH_SIZE 500 → 2000
- InfluxDB write 호출 횟수 감소
- Collector batch 처리 효율 개선
- Kafka backlog 처리 지연 완화

stress 조건에서 k6 유입, Collector consume, InfluxDB 204 응답은 정상인데 Kafka lag가 대량 증가.
→ 원인은 메시지 유입량 대비 InfluxDB write 처리량 부족으로 판단.

파이프라인 구축에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : 수동 Kafka 테스트와 k6 연결 결과 불일치

컨테이너 내부망에서의 메시지 전송은 성공하나, 별도 네트워크를 사용하는 k6 컨테이너에서는 Kafka 브로커 주소 해석 방식이 달라 연결에 실패함.

→ **advertised.listeners** 설정을 내부 통신용(PLAINTEXT)과 외부 통신용(PLAINTEXT_HOST, host.docker.internal:29092)으로 분리하여 적용.

ISSUE 02 : 볼륨 초기화 후 InfluxDB 토큰 무효화

docker compose down -v로 볼륨 초기화 시 InfluxDB의 기존 토큰은 삭제되지만, 연동된 Collector와 Grafana는 구형 토큰을 계속 호출하여 인증 에러가 발생함.

→ 토큰 생성 → .env 반영 → 전체 재기동을 하나의 절차로 문서화.

ISSUE 03 : Grafana 대시보드 Datasource UID 불일치

UI에서 추출한 대시보드 JSON 파일이 기존 Datasource의 고정 UID를 참조하고 있어, Provisioning을 통해 자동 생성된 새 Datasource와 연결되지 않아 패널 데이터가 누락됨.

→ 대시보드 JSON 파일 내의 Datasource 참조 대상을 고정 UID 대신 Provisioning 시 설정한 Datasource 이름 기준으로 동적 매핑되도록 수정.

핵심 학습 포인트

- **파이프라인은 경로 분리가 핵심**

서비스 트래픽과 메트릭 처리를 같은 경로에 두면 장애 원인 격리가 불가능. 분리 설계가 관측의 전제

- **병목은 데이터로 증명**

Kafka consumer group CLI와 Collector 내부 지표를 함께 확인해 병목 위치를 특정하고, 단일 변수(batch size)만 변경해 개선 효과를 수치로 확보

- **Kafka는 완충재**

burst traffic 시 Collector가 처리 속도를 따라가지 못해도 메시지는 유실되지 않고 Kafka lag로 적체.

이후 Consumer가 순차적으로 따라잡을 수 있는 재처리 가능한 구조

- **InfluxDB v3 직접 write**

공식 Kotlin client(v2 기반) 미사용, /api/v3/write_lp HTTP 직접 호출로 의존성 최소화

동일 조건 재측정으로 개선 효과 검증

stress로 병목을 찾고, load 기준으로 Before / After를 고정 측정한 뒤, 동일 stress 조건으로 개선 효과 재검증

검증 흐름

1. stress 병목 탐색

100 → 200 → 500 VUs로 Collector → InfluxDB write 경로 병목 확인

2. load 기준값 기록

Before / After 비교는 동일 조건 유지를 위해 load 시나리오 기준으로 측정

3. 개선 적용

INFLUX_BATCH_SIZE 500 → 2000 변경

4. load 재측정

응답 시간, RPS, 실패율을 동일 조건에서 다시 기록

5. stress 재검증

Kafka lag 감소와 Collector backlog 완화 여부 확인

측정 기준 분리

load

- 목적 : Before / After 기준값 비교
- 조건 : 10 VUs, 30s
- 확인 : avg latency, p95, RPS, failed requests

stress

- 목적 : 파이프라인 병목 확인
- 조건 : 100 → 200 → 500 VUs
- 확인 : Kafka consumer lag, Collector flush, backlog

주의 : Grafana의 kafka_lag는 placeholder.

실제 lag 판단은 Kafka consumer group CLI 기준.

[GitLab](#) · [Docker Compose](#) · [GitHub Actions](#) · [GitLab Runner](#) · [Nginx](#)

온프레미스 Self-Hosted DevOps



목적

- Intel N100 Mini PC에 WSL2 + Docker Desktop 기반 GitLab, Runner, Nginx, Prometheus/Grafana 스택을 구축한 온프레미스 DevOps 플랫폼 구축.
- GitHub Actions Bootstrap과 GitLab Runner CI/CD를 분리해 내부망 운영 구조 검증

Bootstrap → 내부 CI/CD → Reverse Proxy → Monitoring → Backup/Recovery 기준까지 이어지는 운영 흐름 재현

구성 요소

- GitLab Omnibus
- GitHub Actions self-hosted runner
- GitLab Runner Docker Executor
- Nginx HTTPS Reverse Proxy
- Prometheus / Grafana Monitoring

운영 기준

- Container 실행 여부만으로 판단하지 않음
- 접근 경로, Runner, 데이터 보존, 관측 기준까지 검증
- Backup / Recovery 판단 기준 문서화

기획 의도: Cloud 없이 직접 만든 DevOps Lab

구현 언어 / 구성 파일 역할

- **YAML** : Docker Compose, GitHub Actions, GitLab CI/CD, Grafana/Prometheus 설정
- **Shell** : GitLab 운영 점검, 백업·복구 검증, 배포 스크립트
- **Nginx Conf** : Reverse Proxy, HTTPS 내부망 접근 구성 (자체 서명 인증서)

하드웨어 환경

장비	EcoBe-A1 Mini PC
CPU	Intel N100 (4C/4T, TDP 6W)
Memory	16GB
Host OS	Windows 11
Linux	WSL2 Ubuntu
Runtime	Docker Desktop (WSL2 Integration)

Runtime 기준

- Docker Desktop with WSL2 Integration
- WSL2 내부 파일시스템 기반 volume 사용
- /home/user/gitlab 기준 runtime volume 분리

GitLab 리소스 튜닝

- puma worker_processes = 1
- sidekiq concurrency = 5
- postgresql shared_buffers = 256MB
- Docker Desktop Memory 제한

저전력 Mini PC 한 대에서 GitLab과 관측 스택을 모두 운영하는 구조이므로 리소스 제약을 전제로 설계

GitHub Actions와 GitLab Runner 역할 분리

GitLab이 없을 때는 GitHub Actions로 초기 배포하고, GitLab 구축 후에는 GitLab Runner가 내부 Pipeline 실행

Bootstrap 흐름

- 1. Main PC → GitHub Repository Push
- 2. GitHub Actions Workflow 실행
- 3. Mini PC WSL2 self-hosted runner가 outbound로 Job 수신
- 4. Docker Compose로 GitLab / Nginx / Monitoring 기동
- 5. GitLab 구축 후 GitLab Runner가 Repository Pipeline 실행

자동화 흐름 역할 분리

구분	역할	실행 주체	담당 범위
GitHub Actions	GitLab 서버 초기 배포	self-hosted runner	GitLab 서버 초기 배포, 내부망 Mini PC에서 Docker Compose 실행
GitLab Runner	CI/CD Pipeline 실행	Docker Executor	GitLab Repository Pipeline 실행, validate/deploy-check Job 검증

deploy-gitlab.yml 담당 범위

Runtime

GitLab / Runner / Nginx / Monitoring runtime directory 생성

SSL

gitlab.local 자체 서명 인증서 준비

Compose

docker-compose.gitlab.yml과 monitoring compose 기동

Grafana

datasource / dashboard provisioning 파일 복사

검증 기준: runner Idle / Linux / X64, GitLab Web UI 접근, GitLab 주요 서비스 run 상태, Bootstrap과 내부 Pipeline 경로 분리.

Docker Executor 기반 Pipeline 검증

Pipeline 구조 (.gitlab-ci.yml)

```
stages: [validate, deploy-check]
default.tags: [docker, mini-pc, wsl2]

validate-compose
  image: docker:26-cli
  script:
    - docker version
    - docker compose -f infra/compose/docker-compose.gitlab.yml
  config

deploy-readiness-check
  image: alpine:3.20
  script:
    - uname -a
    - test -f infra/compose/docker-compose.gitlab.yml
```

GitLab 서버 자체를 Pipeline으로 재배포하면 순환 의존성 발생
→ 현재는 Compose config 검증 · readiness check만 수행

Pipeline 검증 결과

항목	결과
Pipeline ID	#4
Status	Passed
Job 수	2
실행 시간	17초
validate-compose	Passed
deploy-readiness-check	Passed

Docker Socket 전달 설정

- Runner container에 /var/run/docker.sock 마운트
- config.toml volumes에도 docker.sock 등록
- CI Job Container에서 Host Docker daemon 접근

Nginx HTTPS 접근과 Grafana Provisioning

GitLab Container 포트 직접 접근 경로는 복구용으로 유지하고, 사용자 접근은 gitlab.local HTTPS 경로로 분리

Reverse Proxy 접근 흐름

Main PC

↓ `https://gitlab.local`

Nginx Reverse Proxy

↓ `proxy_pass http://gitlab:80`

GitLab Omnibus Container

복구 경로 : `http://<Mini-PC-IP>:8080`

→ GitLab Container. Nginx 장애 시에도 직접 접근 가능.

Monitoring Provisioning

```
infra/monitoring/grafana/  
├ provisioning/datasources/datasource.yml  
├ provisioning/dashboards/dashboards.yml  
└ dashboards/mini-pc-devops-overview.json
```

```
infra/monitoring/prometheus/  
└ prometheus.yml
```

- gitlab.local hosts 해석 확인
- `https://gitlab.local` GitLab Web UI 접근 확인
- Prometheus target UP 확인
- Grafana Prometheus datasource 자동 연결
- Dashboard provisioning 자동 로드

온프레미스 운영에서 배운 것들

주요 트러블슈팅 3가지

ISSUE 01 : GitHub-hosted runner → Mini PC 내부망 접근 불가

클라우드 환경의 Runner에서 사설망(Mini PC)으로의 Inbound 접근이 불가하여 타임아웃(dial tcp <IP>:22)이 발생함.

→ Mini PC(WSL2) 내부에 Self-hosted Runner를 구성하여, 내부망에서 외부로 GitHub Job을 수신(Outbound)하는 구조로 파이프라인 아키텍처 변경.

ISSUE 02 : WSL2 환경 Docker Credential Helper 오류

WSL2 환경에서 GitHub Actions runner가 Docker 명령 실행 시 인증 헬퍼(Credential Helper)와의 호환성 문제로 실패함.

→ WSL2 환경에 적합하도록 Docker Credential Helper 설정을 재구성하여 인증 및 권한 문제 해결.

ISSUE 03 : 장기 미가동 후 GitLab DB 재초기화

WSL2 및 Docker Desktop 환경의 볼륨 영속성 한계로 인해, 장비 장기 종료 후 재가동 시 DB 스키마가 로드(db:schema:load)되며 데이터가 초기화됨.

→ 해당 환경에서의 영속성 보장 한계를 인지하고, 데이터 보호를 위한 Cold Backup 절차 및 재초기화 대응 기준을 문서화함.

핵심 학습 포인트

- 운영 판단 기준은 수치와 로그로 결정

Container가 실행 중이라도 DB 내부 상태(projects, runners, settings)로 실제 운영 가능 여부를 판단해야 함

- 부트스트랩 자동화의 실용성

GitLab이 없을 때 GitHub Actions로, 있을 때 GitLab Runner로. 두 흐름의 역할을 명확히 분리해야 순환 의존 없이 구성 가능

- 온프레미스의 현실

N100 Mini PC에서 GitLab Omnibus(PostgreSQL, Redis, Gitaly, Rails, Sidekiq)를 모두 올리는 것은 리소스 한계가 존재. 한계 인식 자체가 설계 역량

- Monitoring은 판단을 위한 데이터

Prometheus/Grafana 수치보다 "무엇을 수집하고 무엇을 제외할지" 범위 결정이 핵심 설계

Container 실행 ≠ 운영 가능

Self-hosted 운영은 컨테이너 실행 여부가 아니라 DB 내부 상태, Runner 연결, 접근 경로, 관측 기준, 복구 절차를 함께 확인 필요

정상 판단 기준

- projects / runners / application_settings 유지 여부
- GitLab Web UI 접근 가능
- Runner Pipeline Passed
- Prometheus target UP / Grafana dashboard 로드

운영 금지 명령

- ✗ docker compose down -v
- ✗ docker system prune --volumes
- ✗ rm -rf /home/user/gitlab/data

권장 운영

- ✓ docker compose stop (볼륨 보존)
- ✓ 재배포 전후 mount 경로 확인
- ✓ image 버전 명시 고정

개선 예정

- 정기 backup script 구성
- Restore 후 projects/runners/settings 확인 표준화
- GitLab/Grafana/Prometheus health check script 분리

Phase별 구현 및 검증 완료 현황

Phase별 설명

단계	내용
Phase 1	Mini PC 기본 환경 구성
	WSL2 Ubuntu 설치, Docker Desktop WSL2 Integration, Main PC → Mini PC SSH 접속 확인 (사설 IP : 172.30.1.67)
Phase 2	GitHub Actions 기반 초기 배포
	GitHub-hosted runner 내부망 접근 한계 확인 후 Mini PC에 self-hosted runner 등록. Docker credential helper 오류 대응. Job deploy Succeeded 확인
Phase 3	GitLab 서버 구축
	GitLab Omnibus Container 구성. /home/user/gitlab 기준 runtime volume 분리. root 관리자 로그인, gitlab-ctl status 주요 서비스 run 확인
Phase 4	GitLab Runner 분리 구성
	Docker Executor 기반 Runner 등록. tag: docker/mini-pc/wsl2. 테스트 Pipeline #1 Passed (약 21초) 확인
Phase 5	CI/CD Pipeline 구성
	validate-compose / deploy-readiness-check Job 구성. CI Job Container 안에서 docker version, docker compose config 검증. Pipeline #4 Passed
Phase 6	운영 안정성 검증
	장기 종료 후 GitLab 재기동. projects/runners/application_settings 유지 여부 확인. db:schema:load 로그 기반 DB 재초기화 판단 기준 수립
Phase 7~9	Nginx SSL · Monitoring · Backup
	gitlab.local HTTPS 접근. Prometheus/Grafana provisioning, cold backup 기준 + DB 재초기화 검증 절차 문서화

프로젝트에서 사용한 기술 전체 목록

BACKEND & APPLICATION

분류	기술	사용 프로젝트
Language	Python 3 / Java 17 / Kotlin	01 / 02 / 03
Framework	FastAPI / Spring Boot 3.4.2	01 / 02
Resilience	Resilience4j (CB, Count-based)	02
Template	Jinja2	01
Build	Gradle	02 · 03

OBSERVABILITY & TESTING

분류	기술	사용 프로젝트
Metrics	Prometheus	02 · 04
Metrics Lib	Micrometer	02
Dashboard	Grafana (provisioning)	03 · 04
Tracing	MDC + TracerIdFilter	02
Load Test	k6, Apache JMeter	03 / 02
System Info	psutil, systemctl	01

DATA & MESSAGING

분류	기술	사용 프로젝트
RDBMS	PostgreSQL 16	02
NoSQL	MongoDB	01
Cache	Redis 7 Primary / Caffeine Bucket	02
Messaging	Apache Kafka, xk6-output-kafka	03
Time-Series	InfluxDB v3	03

INFRA & DEVOPS

분류	기술	사용 프로젝트
OS	Rocky Linux 9 / Windows 11 + WSL2	01 / 04
Container	Docker, Docker Compose	전체
SCM	GitLab Omnibus	04
CI/CD	GitHub Actions, GitLab Runner	04
Automation	Ansible	01
	GitHub Actions + Docker Compose	04
Proxy	Nginx (Reverse Proxy, SSL)	01 · 04
Secret	Ansible Vault	01
	Spring Cloud Vault / HashiCorp Vault	02

프로젝트 핵심 키워드 & 성과 요약

프로젝트	핵심 목적	검증 성과	배운 점
01 Server Monitor	OS 환경 차이 직접 검증. 운영 판단형 모니터링	Windows → Docker → Native 3단계 이식 검증 완료. Ansible Vault 기반 운영 자동화	같은 코드, 다른 OS에서 다른 동작. 단계별 격리가 원인 특정의 전제
02 Circuit Breaker	극한 부하에서 시스템 반응 정밀 계측. 방어 기제 실증	CB CLOSED → OPEN 상태 전이 실증. Attack/Normal TG 분리 검증. trace_id 전 구간 추적	장애는 설계로 제어. 불변 설정 없이는 실험 재현성 없음
03 k6 Pipeline	실시간 메트릭 스트리밍 파이프라인 구축 + 병목 개선 실증	end-to-end 파이프라인 검증 완료. Kafka lag 83.2% 감소 수치 확보	병목은 단일 변수 변경 + 수치로 증명. 파이프라인 경로 분리가 관측의 전제
04 Self-Hosted DevOps	온프레미스 환경에서 GitLab 기반 DevOps 플랫폼 직접 구축 및 내부망 CI/CD 운영 구조를 검증	GitHub Actions 기반 초기 배포, GitLab Runner CI/CD, Nginx HTTPS Reverse Proxy, Prometheus/Grafana 모니터링 구성 검증	Self-hosted 운영은 서버 설치뿐 아니라 접근 경로, Runner 실행 위치, 데이터 보존, 관측 기준까지 함께 설계 필요

공통 설계 원칙

직접 검증하고 재현

정상 동작 확인에 그치지 않고 경계 조건·부하 상황·운영 제약을 재현해 시스템 반응을 데이터로 확인

단계별 격리

OS 이식, 부하 증가, 설정 변경 — 항상 단일 변수만 변경해 원인과 결과를 분리

수치로 증명

Kafka lag 83.2% 감소, CB 15건 실패 시 OPEN — 주관이 아닌 측정값과 재현 절차로 결론 도출

한계도 결과물

운영 제약, 이해결 리스크, 재현 조건까지 문서화 — 성공 여부뿐 아니라 한계를 기록하는 것도 검증의 일부

SEO HEE JUNG

Thank you.
For reading.
See you.

저를 더 알고 싶으시다면.

 Notion

 GitHub

Thanks for your time.